

ByteBridge: A Diagnostic Study of Tokenizer-Free Byte Interfaces for Frozen Language Models

Anonymous authors

Paper under double-blind review

Abstract

1 Tokenizer-free byte interfaces promise robustness to spelling noise, rare Uni-
 2 code, multilingual scripts, code, and other inputs that stress fixed subword
 3 vocabularies. A practical path would be to attach a small byte adapter to an
 4 already-trained tokenizer-based language model while keeping the language
 5 model itself frozen. We present ByteBridge, a controlled diagnostic study
 6 of this setting on a frozen Qwen2.5-0.5B causal LM. We evaluate a ladder of
 7 increasingly structured byte interfaces: fixed input-layer byte latents, em-
 8 bedding distillation, token-span reconstruction with fixed, heuristic, and
 9 oracle boundaries, soft prefix injection, and per-layer KV-prefix injection.
 10 All runs use the same train/validation/test manifests, score the same target
 11 tokens as the native tokenizer baseline, freeze the LM and LM head, and
 12 verify zero backbone checksum drift. The result is negative but informa-
 13 tive. Byte adapters reliably improve typo-noise robustness, but they fail to
 14 match native tokenization on clean text and non-typo stress buckets. The
 15 best input-layer adapter reaches $1.84\times$ clean loss relative to the tokenizer
 16 baseline; soft-prefix and all-layer KV-prefix injection improve this to $1.76\times$
 17 and $1.61\times$, respectively, but still miss our $1.5\times$ promising threshold and
 18 never beat the tokenizer baseline outside typo noise. These findings suggest
 19 that frozen tokenizer-based LMs rely on tokenizer-induced boundary, iden-
 20 tity, and positional semantics that small external byte adapters cannot eas-
 21 ily reconstruct. ByteBridge contributes a reproducible negative result and
 22 diagnostic benchmark for future tokenizer-free interfaces for frozen LMs.

23 1 Introduction

24 Subword tokenizers are a central interface between text and modern language models. They
 25 make open-vocabulary modeling tractable, but they also introduce brittle behavior: visually
 26 small edits can change segmentation, rare Unicode characters can expand into long token
 27 sequences, and code-like strings, file paths, identifiers, and multilingual text can be repre-
 28 sented very differently from clean English. Byte-level modeling avoids out-of-vocabulary
 29 behavior and has shown strong robustness and multilingual coverage when the model is
 30 trained for bytes from the start (Xue et al., 2022; Clark et al., 2022; Tay et al., 2022; Yu
 31 et al., 2023). However, most deployed LMs are already trained around a fixed tokenizer.
 32 Retrofitting a tokenizer-free input interface into such a model without updating the model
 33 is appealing, but its feasibility is unclear.

34 This paper asks a deliberately narrow question: can a small byte-derived adapter provide
 35 useful input context to a frozen tokenizer-based LM, while preserving clean-text perfor-
 36 mance and improving robustness on tokenizer-unfriendly inputs? We study this question
 37 with ByteBridge, a set of controlled experiments on Qwen2.5-0.5B (Qwen Team, 2025).
 38 The language model backbone, input embedding table, and LM head remain frozen. Only
 39 byte-side adapter parameters are trained. Output supervision still uses the model’s native
 40 tokenizer, so the study is not a fully tokenizer-free language model; rather, it tests whether
 41 the input side of an existing LM can be replaced or augmented by a byte interface.

42 Our study is motivated by two competing intuitions. The optimistic view is that a byte
 43 adapter can learn to map raw UTF-8 bytes into continuous representations that a frozen

LM can use, especially if those representations are aligned to token embeddings or injected through prefix-tuning-like interfaces (Li and Liang, 2021; Lester et al., 2021). The pessimistic view is that a tokenizer-based LM learns more than an embedding lookup: it learns boundary conventions, token identity structure, positional regularities, and attention patterns induced by its tokenizer. A small external byte adapter may not be able to reproduce this structure.

We evaluate this tension through a diagnostic ladder. We first train fixed-patch input-layer byte latents through `inputs_embeds`. We then test embedding distillation to the Qwen input embedding manifold, token reconstruction pretraining with fixed, heuristic, and oracle token boundaries, byte-derived soft prefixes, and finally per-layer KV-prefix injection using Qwen’s cache API. Each step is designed to answer a specific failure question: Is the problem embedding alignment? token identity? byte boundary segmentation? shallow input injection? The same evaluation suite is reused throughout, covering clean English, typo noise, Unicode stress, multilingual text, code, and tokenizer-stress strings.

The main result is negative. Byte-side adapters consistently beat the tokenizer baseline on typo-noise buckets, where the native tokenizer sees corrupted prompts and the byte adapter learns a denoising-style context. But every adapter remains substantially worse on clean text, and the native tokenizer baseline continues to dominate Unicode, multilingual, code, and tokenizer-stress categories. Deeper injection helps: soft prefix p64 improves Unicode/code/stress losses over input replacement, and all-layer KV-prefix achieves the best clean retention, $1.61\times$ the tokenizer baseline. Yet this still fails to reach our pre-registered promising threshold of $\leq 1.5\times$ and remains far from a strong pass threshold of $\leq 1.2\times$.

Our contributions are:

- We introduce a controlled benchmark and evaluation protocol for input-side tokenizer-free adapters on frozen tokenizer-based LMs, with paired conditional targets and fixed splits.
- We provide a phase-by-phase diagnostic study of byte adapters, including input replacement, embedding distillation, token reconstruction, boundary-aware patching, soft prefixes, and KV-prefix injection.
- We report a reproducible negative result: under the tested frozen-Qwen2.5-0.5B, single-GPU, small-adapter setting, byte-derived adapters improve typo robustness but do not become competitive replacements for native tokenized input.

2 Problem Setting

Let M be a pretrained causal LM with tokenizer T , input embedding matrix E , transformer F , and LM head H . In all experiments, E , F , and H are frozen. Given an input string x and target string y , the native tokenizer baseline computes

$$\mathcal{L}_{\text{tok}}(x, y) = -\sum_t \log p_M(y_t \mid T(x), y_{<t}), \quad (1)$$

where y_t are target tokens under the same tokenizer. The byte-interface variants replace or augment the conditioning on x with byte-derived continuous states $A(b(x))$, where $b(x)$ is the UTF-8 byte sequence and A is a trainable adapter. The target side remains tokenized so that all systems are scored on exactly the same target tokens.

This paired conditional setup is important. Tokenizer-free adapter evaluations can be misleading if one system is scored on bytes and another on tokens, or if noisy inputs change the supervised target. We instead use the same target tokens for all models. For noisy examples, x is the perturbed string and y is the clean target suffix. Loss is always computed only over target tokens, not over prefix or adapter positions.

The primary metric is per-bucket target-token loss. We also save perplexity in experiment artifacts, but focus on loss because several stress buckets yield very large perplexities. To measure clean-text degradation, we use

$$\text{clean retention}(A) = \frac{\mathcal{L}_A(\text{clean})}{\mathcal{L}_{\text{tok}}(\text{clean})}. \quad (2)$$

Before running the main diagnostic ladder, we set `clean retention` ≤ 1.5 as a “promising” threshold and `clean retention` ≤ 1.2 as a strong clean-retention threshold. We also record whether each run beats the tokenizer baseline per bucket, trainable parameter counts, throughput, peak GPU memory, and checksum drift of frozen model tensors.

3 Evaluation Suite

We construct a small, reproducible suite intended to stress tokenizer-unfriendly inputs while remaining feasible on a single RTX 5090. The train/validation/test manifests are fixed after construction and reused across all phases. Each manifest record contains an identifier, bucket, input text, target text, source, perturbation metadata, seed, and clean-example identifier where applicable. Test examples are never used for training.

The test buckets are:

- Clean English: short natural-language, Wikipedia-like, instruction-like, and QA-like snippets.
- Typo noise: clean English with random deletion, insertion, substitution, adjacent swaps, repeated characters, whitespace perturbations, and casing noise at light, medium, and heavy levels.
- Unicode stress: emoji, accented characters, combining marks, zero-width joiners, full-width characters, mathematical symbols, punctuation variants, and normalized/non-normalized mixtures.
- Multilingual text: short Chinese, Japanese, Korean, Arabic, Hindi, Russian, and Latin-script accented examples.
- Code: Python, JavaScript, JSON, shell commands, stack traces, regexes, SQL, and configuration snippets.
- Tokenizer-stress strings: URLs, file paths, package names, UUIDs, base64-like and hexadecimal strings, logs, long identifiers, and mixed separators.

The native tokenizer baseline is a frozen Qwen2.5-0.5B evaluated directly on the paired conditional test set. Its clean-English loss is 0.3089. This baseline is intentionally strong: it uses the same tokenizer and embedding conventions under which the model was pretrained.

4 Diagnostic Adapter Ladder

4.1 Input-layer byte latents

The simplest ByteBridge adapter maps fixed-size byte patches into L latent vectors in the LM hidden dimension. These latents are passed through `inputs_embeds` as the prompt representation. This directly tests whether a small adapter can replace tokenized prompt embeddings at the input layer. We train clean-only and noise-augmented variants; the strongest Phase 2 run uses $L = 32$ and noise-augmented training for 2000 steps.

4.2 Embedding distillation

If input-layer latents fail because they are off the Qwen embedding manifold, a natural remedy is to align them to frozen token embeddings. We train the byte adapter with two distillation objectives: fixed token-span alignment, where token embeddings are truncated or pooled to L spans, and pooled sequence alignment, where byte and token representations are mean/max pooled. The adapter is then fine-tuned with the same frozen-LM conditional objective.

4.3 Token reconstruction and boundary-aware patching

Embedding MSE may be too weak: the LM may need token identity and boundary structure, not just nearby continuous vectors. We therefore build a token-byte mapping tool

Stage	Representative run	clean loss	clean retention	wins vs. tokenizer
Tokenizer	native Qwen tokenizer	0.3089	1.00	–
Phase 2	fixed input, noise, $L = 32$	0.6770	2.19	typo L/M/H
Phase 3	distill then LM, $L = 32$	0.6863	2.22	typo L/M/H
Phase 4	fixed recon then LM	0.5669	1.84	typo L/M/H
Phase 5	soft prefix, $P = 64$	0.5441	1.76	typo L/M/H
Phase 6	KV prefix, $P = 32$, all layers	0.4985	1.61	typo L/M/H

Table 1: Best representative runs across the diagnostic ladder. “Typo L/M/H” denotes the light, medium, and heavy typo-noise buckets. No byte-interface run beats the tokenizer baseline outside typo noise.

that maps tokenizer tokens to UTF-8 byte spans. For Qwen on this suite, tokenizer offset mapping yields zero unmapped tokens. We test three boundary modes: fixed byte patches, heuristic tokenizer-free boundaries based on UTF-8/script/punctuation/whitespace changes, and oracle tokenizer-derived byte spans. The oracle mode is a diagnostic upper bound, not a tokenizer-free method.

During reconstruction pretraining, a trainable classifier predicts tokenizer token IDs from byte/span latents with cross-entropy loss. The classifier head is discarded for LM evaluation; only the adapter initializes the frozen-LM conditional training run.

4.4 Soft prefix injection

Input replacement may be too brittle because it forces byte latents to behave like token embeddings at every prompt position. Soft prefix injection instead compresses the input bytes into P learned prefix embeddings, concatenates them before tokenized target embeddings, and computes loss only on target positions:

$$\text{inputs_embeds} = [A_{\text{prefix}}(b(x)); E(T(y_{<t}))]. \quad (3)$$

This preserves native target-side teacher forcing while giving the frozen LM byte-derived context.

4.5 KV-prefix injection

Finally, we test whether byte-derived information must be injected into deeper layers. We probe Qwen’s DynamicCache API and construct byte-derived key/value tensors with shape $[B, H_{kv}, P, d_h]$ for selected layers. Qwen2.5-0.5B uses 24 layers, hidden size 896, 14 attention heads, 2 key/value heads, and head dimension 64. We test first-4-layer and all-layer injection. For first- N runs, non-injected layers receive zero K/V tensors of the same prefix length to keep cache sequence lengths consistent across layers; this is a diagnostic compromise, not an optimal prefix-tuning design.

5 Results

5.1 Main comparison

Table 1 summarizes the best representative run from each phase. The pattern is consistent: each later diagnostic improves at least one aspect of the previous interface, but none approaches native tokenizer conditioning on clean text. The best input-layer reconstruction run reaches $1.84\times$ clean retention. Soft prefixes improve this to $1.76\times$, and all-layer KV-prefix reaches $1.61\times$. All remain above the $1.5\times$ promising threshold.

5.2 Bucket-level behavior

Table 2 shows selected bucket losses. The tokenizer baseline remains strongest on clean English, code, tokenizer-stress, Unicode, and multilingual text. The byte adapters are

Bucket	tokenizer baseline	Phase 2 best	Phase 4 fixed recon	soft prefix p64	KV prefix p32 all
clean English	0.3089	0.6770	0.5669	0.5441	0.4985
code	0.3876	2.4261	2.0058	0.7878	1.3462
tokenizer stress	0.4806	3.1706	2.8533	1.5118	1.5969
Unicode stress	0.5614	3.8226	2.3900	0.9630	0.9809
typo light	1.3838	0.7454	0.6416	0.5817	0.5167
typo medium	1.8428	0.7786	0.6626	0.6097	0.5298
typo heavy	2.8353	0.8203	0.7749	0.6686	0.5986
multilingual zh	0.6085	4.5419	4.7134	4.9432	11.1217
multilingual ar	0.3806	3.9973	4.4678	5.4298	10.7503

Table 2: Selected test losses. Lower is better. Byte-derived adapters consistently help typo-noise buckets but remain worse than native tokenization on clean and non-typo stress categories.

strongest on typo noise. This is the one robust positive signal across phases: when the prompt is corrupted but the target is clean, byte adapters can learn a denoising-style context that degrades less than native tokenized conditioning.

Deeper injection helps non-typo stress relative to earlier byte variants. Soft prefix p64 reduces code loss from 2.0058 under fixed reconstruction to 0.7878, tokenizer-stress from 2.8533 to 1.5118, and Unicode from 2.3900 to 0.9630. However, these losses remain worse than tokenizer baseline losses of 0.3876, 0.4806, and 0.5614. All-layer KV-prefix gives the best clean loss but is worse than soft prefix p64 on code and tokenizer-stress, and multilingual losses remain very poor.

5.3 What each diagnostic rules out

Fixed byte patches are insufficient. The Phase 2 input-layer adapter confirms engineering feasibility: gradients flow through inputs_embeds, adapter loss decreases, LM trainable parameters remain zero, and checksum drift is zero. But the best Phase 2 clean retention is $2.19\times$, with severe degradation on Unicode, code, multilingual, and tokenizer-stress buckets. This rules out the simplest fixed-patch interface as a competitive tokenizer replacement.

Embedding alignment is not enough. Projection alignment works as an auxiliary objective: distillation loss drops from 0.9583 to 0.0637 for $L = 32$ and from 0.9654 to 0.0752 for $L = 64$. Downstream LM evaluation does not improve: the $L = 32$ distill-then-LM run has $2.22\times$ clean retention. This suggests that proximity to the embedding manifold under MSE/cosine objectives does not recover the discrete identity and sequence structure the frozen LM expects.

Token identity is learnable, but not sufficient. Token reconstruction gives a stronger diagnostic. Fixed reconstruction reaches validation clean top-1 accuracy 0.8669; oracle-boundary reconstruction reaches 0.9945. This confirms that token identity can be recovered from byte spans, especially when token boundaries are known. Yet downstream LM clean retention remains poor: fixed reconstruction improves to $1.84\times$, while oracle-boundary reconstruction is $2.20\times$. Oracle boundaries improve Unicode/code/stress losses relative to Phase 2, but they do not repair clean-text conditioning. Therefore boundary and identity errors are real bottlenecks, but not the only bottlenecks.

Deeper injection helps, but not enough. Soft prefix and KV-prefix variants test whether the frozen LM needs byte information as attention context rather than replacement input embeddings. The answer is partially yes. Soft prefix p64 dramatically improves code, Unicode, and tokenizer-stress losses over input-layer variants. All-layer KV-prefix gives the best clean retention, $1.61\times$, and preserves typo-noise gains. But neither reaches $1.5\times$ clean retention, and neither beats the tokenizer baseline on non-typo stress buckets.

6 Discussion

The negative result is informative because each failed variant addresses a plausible explanation for the previous failure. If the issue were simply that byte latents looked unlike token embeddings, embedding distillation should have improved downstream loss. It did not. If the issue were only boundary segmentation, oracle token boundaries should have substantially closed the clean retention gap. They did not. If the issue were only shallow input injection, all-layer KV-prefix should have approached native token conditioning. It improved clean retention but remained far from the tokenizer baseline.

Our interpretation is that frozen tokenizer-based LMs depend on a bundle of tokenizer-induced conventions. Native token embeddings encode not only local text content, but also the distribution of token identities, boundary lengths, position increments, and attention patterns seen during pretraining. A small byte adapter trained after the fact can learn useful robust cues, especially for typo-noise denoising, but it does not recreate the full conditioning interface.

This does not imply that byte-level modeling is ineffective. Models trained from scratch or heavily adapted for bytes can be strong (Xue et al., 2022; Clark et al., 2022; Yu et al., 2023). Nor does it rule out larger adapters, longer training, contrastive retrieval-style objectives, partial LM finetuning, or architectures designed to bridge bytes and tokens. It specifically rules against a tempting practical shortcut: attach a small byte adapter to a frozen tokenizer-based LM and expect it to replace native tokenized prompts.

7 Related Work

Byte and character-level language models. ByT5 removes text preprocessing and operates directly on UTF-8 bytes, improving robustness and multilingual coverage in an encoder-decoder setting (Xue et al., 2022). CANINE models Unicode characters with downsampling and upsampling to reduce sequence length (Clark et al., 2022). Charformer learns gradient-based subword tokenization within the model (Tay et al., 2022). MEGABYTE uses multiscale byte modeling with patch-level structure for long byte sequences (Yu et al., 2023). These systems differ from ByteBridge because they train architectures for byte or character inputs, whereas we freeze an existing tokenizer-based LM and train only an external input adapter.

Tokenization and robustness. Prior work has shown that tokenization choices affect robustness, fairness, multilingual efficiency, and downstream performance (Bostrom and Durrett, 2020; Rust et al., 2021; Mielke et al., 2021). ByteBridge focuses on a complementary question: whether an already-tokenized LM can be retrofitted with a tokenizer-free input side while keeping the LM frozen.

Parameter-efficient frozen-LM adaptation. Prompt tuning, prefix tuning, adapters, and LoRA demonstrate that frozen or mostly frozen LMs can be steered by small trainable modules (Li and Liang, 2021; Lester et al., 2021; Houlsby et al., 2019; Hu et al., 2022). Our soft-prefix and KV-prefix experiments borrow this intuition, but the task is harder: the adapter must encode arbitrary raw input bytes into context that substitutes for tokenized prompt information, rather than adapt a task using the model’s native tokenizer.

8 Limitations

This study is scoped to Qwen2.5-0.5B, small adapters, a single RTX 5090 environment, short runs of roughly 1k–2k steps for the main diagnostics, and a synthetic/curated stress benchmark. We do not claim that all byte interfaces for all frozen LMs must fail. We also do not eliminate the target-side tokenizer: supervision and output scoring use Qwen target tokens to make comparisons fair. The study has not yet tested larger models, multiple tokenizer families, substantially larger adapters, long training schedules, contrastive pretraining with negatives, or partial LM finetuning.

The oracle-boundary reconstruction experiments use tokenizer-derived boundaries and therefore are diagnostic upper bounds, not tokenizer-free systems. KV-prefix experiments use a practical cache construction for Qwen’s installed Transformers version; first- N -layer variants use zero K/V tensors for non-injected layers to maintain cache length consistency, which may not be optimal.

9 Conclusion

ByteBridge asks whether a small tokenizer-free byte adapter can replace or augment native tokenized input for a frozen tokenizer-based LM. Across a controlled ladder of increasingly expressive interfaces, the answer under our tested setting is no. Byte adapters produce a consistent typo-noise robustness signal and deeper injection improves several stress categories, but clean retention remains too poor and native tokenization continues to dominate non-typo stress buckets. The result suggests that frozen LMs inherit strong dependence on tokenizer-induced token identity, boundary, and positional semantics. Future tokenizer-free retrofitting methods likely need stronger objectives, larger or deeper adapters, partial model adaptation, or architectures trained with byte interfaces from the beginning.

References

- Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of EMNLP*, 2020.
- Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 2022.
- Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Larousilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *ICML*, 2019.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *EMNLP*, 2021.
- Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *ACL*, 2021.
- Sabrina J. Mielke, Zaid Alyafeai, Elizabeth Salesky, Colin Raffel, Manan Dey, Matthias Gallé, Arun Raja, Chenglei Si, Wilson Y. Lee, Benoît Sagot, and Samson Tan. Between words and characters: A brief history of open-vocabulary modeling and tokenization in NLP. In *ACL*, 2021.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? On the monolingual performance of multilingual language models. In *ACL*, 2021.
- Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *ICLR*, 2022.
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 2022.
- Lili Yu, Daniel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *NeurIPS*, 2023.

302 A Reproducibility Artifacts

303 All experiments save configuration files, environment metadata, training metrics, sum-
304 maries, checkpoints, and per-bucket evaluation outputs. Frozen model trainable parameter
305 counts are zero in all reported LM runs, and frozen checksum delta is zero. The Phase 2
306 train/validation/test manifests are reused unchanged from Phase 2 through Phase 6.

307 B KV Cache Probe Details

308 Qwen2.5-0.5B in the tested Transformers environment uses a DynamicCache. Each layer's
309 key and value tensors have shape $[B, 2, S, 64]$, reflecting grouped-query attention with 14
310 attention heads and 2 key/value heads. Manual DynamicCache construction works; legacy
311 tuple-style `past_key_values` does not work in this environment because the model expects
312 cache objects with `get_seq_length`.